

# CubeVision



Rubik's Cube State Reconstruction  
& Solving Using Computer Vision

---

CS4391 Introduction to Computer Vision | Jacob Viveros, Carlos Esquivel, Farzaan Ali, Nicholas Watts | Spring 2026

Data Collection

Labeling

YOLOv8n Training

TensorRT

Jetson Deploy

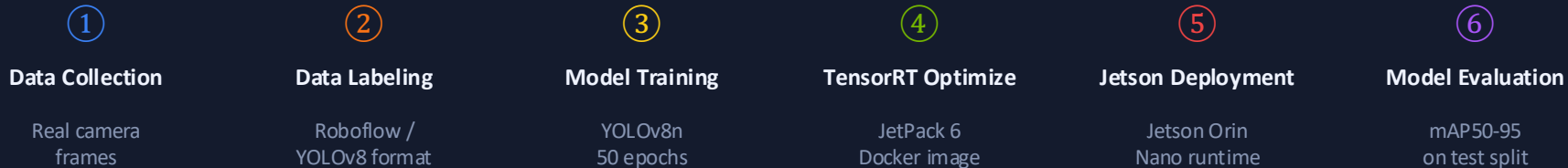
Evaluation

# • System Pipeline



*Data Flow*

## 6 Required Project Milestones



# • Data Collection & Labeling

## Real Camera Capture

- Live webcam frames on PC and Jetson Orin Nano
- 6 faces scanned in fixed order:  $U \rightarrow R \rightarrow F \rightarrow D \rightarrow L \rightarrow B$
- Program waits until exactly 9 stickers detected before allowing save
- Center tile color identifies each face (never moves on a real cube)
- Duplicate face detection: if center color already captured  $\rightarrow$  rejected
- All 6 face grids serialized  $\rightarrow$  cube\_faces.json

## Roboflow Data Labeling

|                   |                                |
|-------------------|--------------------------------|
| <b>Workspace:</b> | UTD                            |
| <b>Project:</b>   | rubik-s-cube-sticker-detection |
| <b>Version:</b>   | v1                             |
| <b>Format:</b>    | YOLOv8                         |
| <b>Labels:</b>    | sticker (single class)         |

---

Bounding boxes drawn around each individual sticker on every frame of the Rubik's Cube.  
Dataset downloaded via Roboflow Python SDK:

```
version.download("yolov8")
```

# ● Model Training — YOLOv8n

## Training Configuration

model yolo8n.pt (nano)

epochs 50

imgsz 640 × 640 px

batch 16

device 0 (GPU / CUDA)

conf 0.5 threshold

```
model.val(split='test')  
→ prints mAP50-95
```

## YOLO: Key Formulas Used

Intersection over Union (IoU) — bounding box regression:

$$\text{IoU}(\hat{B}; B) = |\hat{B} \cap B| / |\hat{B} \cup B|$$

Non-Maximum Suppression (NMS) — remove duplicate boxes:

Keep box  $B$  if:  
 $\text{conf}(B) \geq 0.5 \quad \wedge \quad \text{IoU}(B, B^*) < \tau$

Sticker Square Validation (tiledetection.py):

$\theta(p, q, r) = \arccos( |v_1 \cdot v_2| / (|v_1| \cdot |v_2|) )$   
where  $v_1 = p - q$ ,  $v_2 = r - q \rightarrow$  corner angle

All 4 corner angles must satisfy:  $|\theta - 90^\circ| \leq 20^\circ$

All 4 sides within 60% of longest side

# ● HSV Color Classification & 3×3 Grid Reconstruction

## HSV Color Classifier

Step 1 — Crop center 50% of detected ROI:

```
ROI_c = ROI[ h/4:3h/4, w/4:3w/4 ]
```

Step 2 — Compute mean HSV of crop:

```
(H, S, V) = mean( HSV( ROI_c ) )
```

Step 3 — Classify via HSV thresholds:

$S < 40 \wedge V > 120 \rightarrow \text{"white"}$

$20 \leq H \leq 40 \rightarrow \text{"yellow"}$

$8 \leq H < 20 \rightarrow \text{"orange"}$

$H < 8 \vee H \geq 170 \rightarrow \text{"red"}$

$90 \leq H \leq 130 \rightarrow \text{"blue"}$

$45 \leq H \leq 89 \rightarrow \text{"green"}$

## 3×3 Grid Reconstruction

INPUT:

9 detections {cx, cy, label}

1. Sort cy → find 2 largest gaps

→ 3 row clusters

→  $\text{row\_center}[0..2] = \text{mean}(\text{row})$

2. Assign each detection:

$\text{row} = \text{argmin}_i |cy - \text{row\_center}[i]|$

3. Sort each row by cx (left→right)

OUTPUT: 3×3 grid of color labels

$\text{center} = \text{grid}[1][1] = \text{face ID}$

# ● Cube Solving — Kociemba's Two-Phase Algorithm

## Building the 54-Character String

1. Map center tile color → face letter

```
center of U-face = white  →  white: 'U'  
center of R-face = red    →  red:  'R'
```

2. Concatenate stickers in URFDLB order

```
6 faces × 9 stickers (row-major)  
= 54-character cube string
```

3. Validate: 6 colors × 9 each, no unknowns

```
assert len(cube_string) == 54
```

Example solved-state string:

```
"UUUUUUUUURRRRRRRRFFFFFFF  
FFDDDDDDDDLLLLLLLLBBBBBBBB"
```

## Kociemba Two-Phase Solver

### Phase 1 — Reduce to subgroup $G_1$ :

Orient all edges & corners so only  
U and D face moves are needed

### Phase 2 — Solve within $G_1$ :

Use only  $\frac{1}{2}$ -turn moves on all faces  
(U2, D2, F2, B2, L2, R2) to reach solved

---

Example terminal output:

```
$ python solve_cube.py --input cube_faces.json
```

```
Kociemba string: UURUFUUUU...
```

```
Solution:  R U R' U' F2 L D2
```

```
1. R    2. U    3. R'   4. U'   5. F2
```

# TensorRT Optimization & Jetson Orin Nano Deployment

## NVIDIA Docker Container

```
IMAGE: ultralytics/ultralytics:  
latest-jetson-jetpack6
```

Key Docker flags:

`--runtime=nvidia`                      Enable NVIDIA GPU runtime

`--privileged`                      Full device access

`--device /dev/video0`              Camera passthrough

`--network=host`                    Native network stack

TensorRT is bundled inside the Ultralytics  
JetPack 6 Docker image → automatic INT8/FP16  
optimization for Jetson GPU

## Full Deployment Pipeline

1

```
sh run_ultralytics_jetson.sh
```

Launch NVIDIA Docker container

2

```
python jetson_detection.py  
--model best.pt
```

Live YOLO inference on Jetson cam

3

```
python capture_cube.py  
--model best.pt
```

Scan all 6 faces → cube\_faces.json

4

```
python solve_cube.py  
--input cube_faces.json
```

Compute & print solution moves

# ● Evaluation & Results

Detection Metric

## mAP50-95

COCO-style AP across  
IoU 0.50→0.95

Confidence Threshold

## conf $\geq$ 0.5

Minimum per-sticker  
detection score

Sticker Requirement

## 9 / 9

Exact match required  
per face before save

### Evaluation & Validation Pipeline

Train → `model.val(split='test')` → mAP50-95 printed to console

**Cube-state validation:** 54 stickers total | 6 colors × 9 each | unique center tiles | no 'unknown' labels